Ch4. 업무자동화 프로젝트 사례 - 작성중

** 작성 예정 프로젝트 리스트

1. 웹크롤링 편

1) 특정 주제 네이버 뉴스 크롤링

2) 원하는 사이트 이미지/파일 다운

3) 유튜브 댓글 크롤링, 좋아요 누르기

4) 잡플래닛 로그인&회사 리뷰 크롤링

2. OS/폴더 편

1) 파일 자동 분류(제목, 확장자 등)

2) 파일명 바꾸기(넘버링)

3) Outlook 이메일 자동 보내기

3. 엑셀편

1) 엑셀 데이터 자동 분류

2) 같은 양식 데이터 취합하기/수정하기

3) 엑셀 파일 PDF 변환하기

4) 엑셀 시트 분리하기

Outlook 첨부 엑셀 파일 데이터 자동 취합하기

1. 프로그램 컨셉

위 프로그램은 사내 Outlook(전자메일)을 활용하면서 메일의 자료를 주기적으로 취합해야하는 사용자에게 유용합니다. 예를 들면, 메일로 오는 발주서, 수주 양식 파일들을 자동으로 다운받고 엑셀로 취합해야 하는 경우 활용할 수 있습니다.

Concept1. Outlook(전자메일)을 사용하고 아래와 같이 메일에 첨부 엑셀 파일이 있는 경우 활용할 수 있습니다.

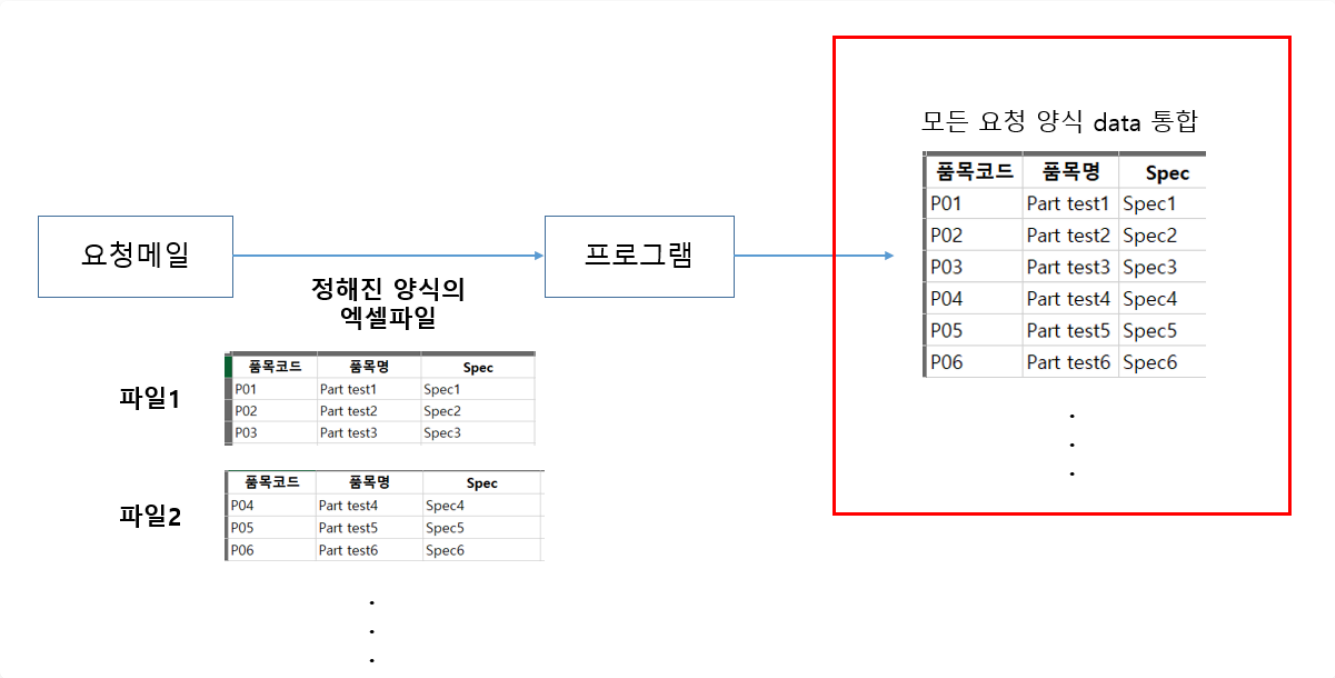


Concept2. 첨부 엑셀 파일의 데이터를 아래 엑셀과 같이 자동으로 한개 엑셀파일로 취합합니다.

	A	B	C	D	E
1		**품목코드**	**품목명**	**Spec**	
2	**0**	P01	Part test1	Spec1	
3	**1**	P02	Part test2	Spec2	
4	**2**	P03	Part test3	Spec3	
5	**3**	P04	Part test4	Spec4	
6	**4**	P05	Part test5	Spec5	
7	**5**	P06	Part test6	Spec6	
8	**6**	P07	Part test7	Spec7	
9	**7**	P08	Part test8	Spec8	
10	**8**	P09	Part test9	Spec9	
11	**9**	P10	Part test10	Spec10	
12	**10**	P11	Part test11	Spec11	
13	**11**	P12	Part test12	Spec12	
14					
15					

위 설명의 개념도는 아래와 같습니다.

※ 프로그램 개념도



2. 사용 라이브러리 정보

이번 챕터에서는 win32com.client과 os 모듈을 활용합니다.

- win32com.client : 이메일 제어 및 첨부파일 데이터 읽어오기(Ch3-3 참고)
- os 모듈 : 폴더 안의 파일 정보를 읽어올 때 사용 (Ch3-4 참고)

3. 전체 코드

코드에 대한 설명은 주석으로 달아놓았으니 아래 내용을 참고해주시기 바랍니다.

```
import win32com.client
import os
import pandas as pd

#outlook 정해진 폴더에서 첨부파일을 다운로드 하는 함수
#flodername : outlook 폴더명, downpath : 다운받을 위치
def att_down(foldername, downpath):
    outlook = win32com.client.Dispatch("Outlook.Application").GetNamespace("MAPI")
    inbox = outlook.GetDefaultFolder(6).Folders[foldername]
    messages = inbox.Items

    no = 1 #중복 파일명이 있을 것을 대비해 추가한 파일명 인덱스
    m_count = 1 #메일 카운트
    #각 메일마다 첨부파일을 다운로드 하는 루프
    for ms in messages:

        attachments = ms.Attachments #해당 메일의 첨부파일 개수(integer)
        r = attachments.count
        print(str(m_count)+"번째 메일 첨부파일 개수 : ",str(r)+"개")
        m_count=m_count+1
        for i in range(1, r + 1):
            attachment = attachments.Item(i)
            attachment.SaveASFile(downpath + str(no) + "_" + str(attachment))  # 파일명을 넣어
야 함.
            no = no + 1


def att_df(downpath):
    filelist = os.listdir(downpath)
    df_list = []

    #각각 파일을 데이터프레임화해서 리스트에 넣기
    for fn in filelist:
         df = pd.read_excel(downpath+"/"+fn, engine="openpyxl")
         df.dropna(inplace =True)
         df_list.append(df)
    print(df_list)

    return df_list

def att_merge(df_list):
    count = len(df_list)
```

```
    if count == 1:
        result = df_list[0]

    elif count == 2:
        result = pd.concat([df_list[0], df_list[1]], ignore_index=True)

    else:
        temp = df_list[0]
        for i in range(1, count):
            result = pd.concat([temp, df_list[i]], ignore_index=True)
            temp = result

    print(result)
    result.to_excel("outlook결과파일.xlsx")


###첨부파일 다운로드 하기
fn = "받은편지함 폴더명"
downpath = "첨부파일 다운받을 경로"
att_down(fn, downpath)
df_list = att_df(downpath)
att_merge(df_list)
```

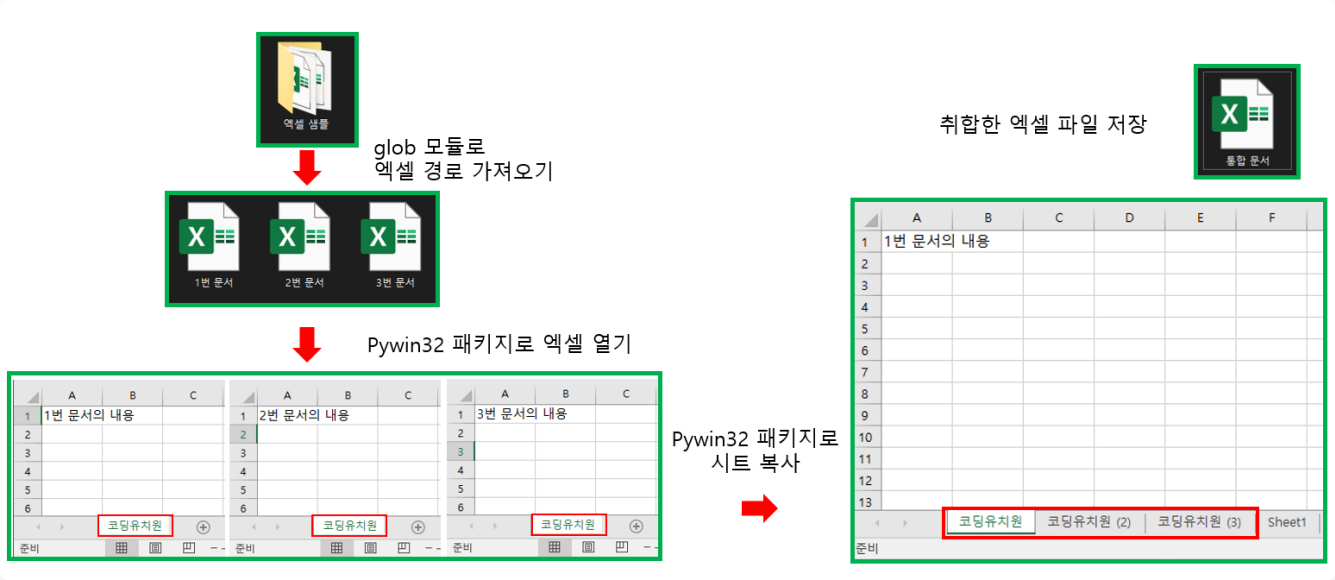
개별 파일의 엑셀 시트 새 파일에 취합하기

1. 프로그램 컨셉

오늘은 제목과 같이 개별 엑셀 파일의 특정 시트들을 새로운 엑셀 파일에 모두 합치는 파이썬 프로그램을 만들어 보겠습니다.

수십 수백개의 엑셀 파일의 특정 시트들을 보고싶거나 인쇄할 때 한곳으로 취합한다면 훨씬 시간을 절약할 수 있겠죠?

대략적인 컨셉은 다음과 같습니다.



2. 사용 라이브러리 정보

- Pywin32 (엑셀 어플리케이션을 다루기 위함)
- glob (폴더 내의 엑셀 파일의 경로를 가져오기 위함)

3. 전체 코드

이해하기 쉽도록 자세하게 주석을 달아두었으니 어렵지 않을거예요. 만약 이해가지 않으시는 부분은 댓글로 질문해주세요!

```
# step1.관련 모듈 및 패키지 import
import glob
import win32com.client

# step2.win32com(pywin32)를 이용해서 엑셀 어플리케이션 열기
excel = win32com.client.Dispatch("Excel.Application")
excel.Visible = True #실제 작동하는 것을 보고 싶을 때 사용

# step3.엑셀 어플리케이션에 새로운 Workbook 추가
wb_new = excel.Workbooks.Add()

# step4.glob 모듈로 원하는 폴더 내의 모든 xlsx 파일의 경로를 리스트로 반환
list_filepath = glob.glob(r'C:\Users\SANGWOO\Desktop\VSCODE\엑셀 샘플\*.xlsx', recursive=True)

# step5.엑셀 시트를 추출하고 새로운 엑셀에 붙여넣는 반복문
for filepath in list_filepath:

    # 받아온 엑셀 파일의 경로를 이용해 엑셀 파일 열기
    wb = excel.Workbooks.Open(filepath)

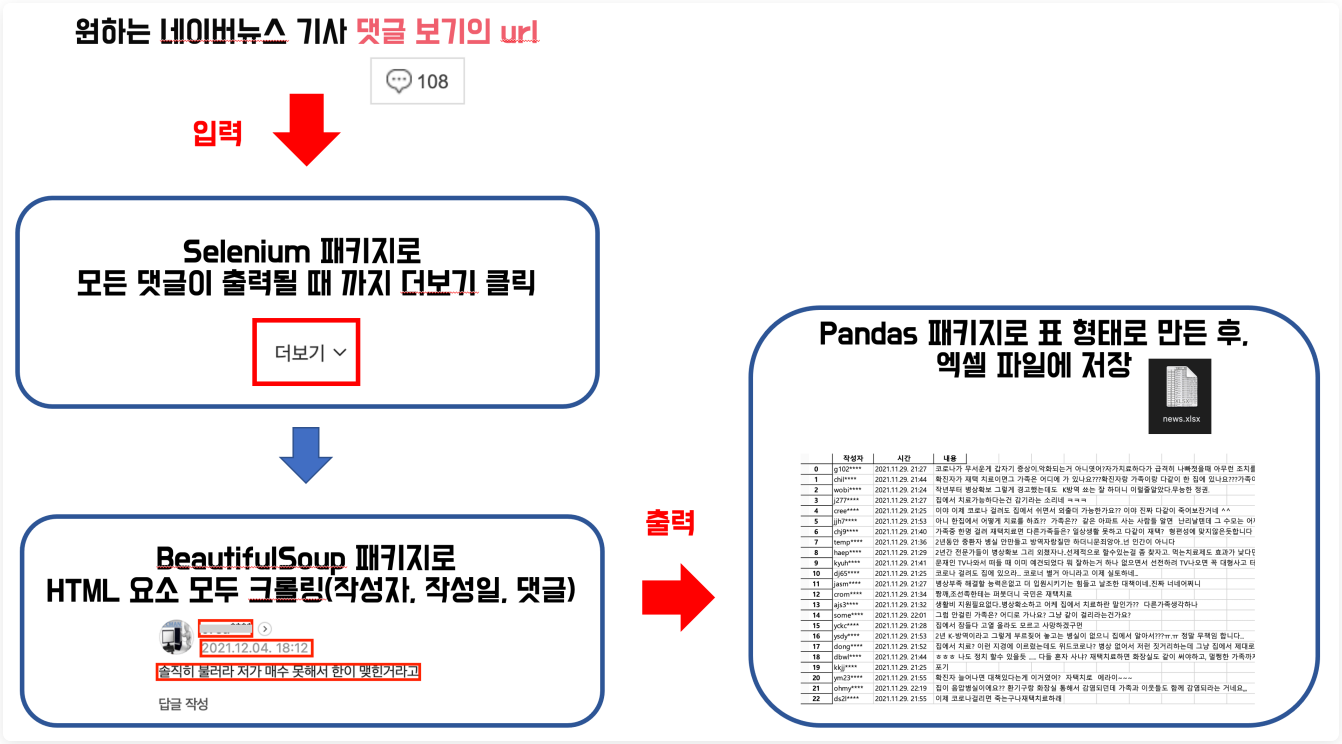
    # 새로 만든 엑셀 파일에 추가
    # 추출할wb.Worksheets("추출할 시트명").Copy(Before=붙여넣을 wb.Worksheets("기준 시트명")
    wb.Worksheets("코딩유치원").Copy(Before=wb_new.Worksheets("Sheet1"))

# step6. 취합한 엑셀 파일을 "통합 문서"라는 이름으로 저장
wb_new.SaveAs(r"C:\Users\SANGWOO\Desktop\VSCODE\엑셀 샘플\통합 문서.xlsx")

# step7. 켜져있는 엑셀 및 어플리케이션 모두 종료
excel.Quit()
```

네이버뉴스 댓글 수집해서 엑셀로 저장하기

1. 프로그램 컨셉



2. 사용 라이브러리 정보

- Selenium (동적 수집, 버튼 조작을 위함)
- BeautifulSoup (정적 수집, 빠른 수집을 위함)
- Pandas (수집한 정보의 리스트를 표 형태로 만들고, 엑셀로 저장하기 위함)

3. 전체 코드

이해하기 쉽도록 자세하게 주석을 달아두었으니 어렵지 않을거예요.

```
# step1. 관련 패키지 및 모듈 불러오기
from selenium import webdriver
from  selenium.webdriver.common.by  import  By
import time
import pandas as pd
from bs4 import BeautifulSoup


# step2. 네이버 뉴스 댓글정보 수집 함수
def get_naver_news_comments(url, wait_time=5, delay_time=0.1):

    # 크롬 드라이버로 해당 url에 접속
    driver = webdriver.Chrome()

    # (크롬)드라이버가 요소를 찾는데에 최대 wait_time 초까지 기다림 (함수 사용 시 설정 가능하며 기본값은 5초)
    driver.implicitly_wait(wait_time)

    # 인자로 입력받은 url 주소를 가져와서 접속
    driver.get(url)

    # 더보기가 안뜰 때 까지 계속 클릭 (모든 댓글의 html을 얻기 위함)
    while True:

        # 예외처리 구문 - 더보기 광클하다가 없어서 에러 뜨면 while문을 나감(break)
        try:
            more  =  driver.find_element(By.CLASS_NAME,  'u_cbox_btn_more')
            more.click()
            time.sleep(delay_time)

        except:
            break

    # 본격적인 크롤링 타임

    # selenium으로 페이지 전체의 html 문서 받기
    html = driver.page_source

    # 위에서 받은 html 문서를 bs4 패키지로 parsing
    soup = BeautifulSoup(html, 'lxml')

    # 1)작성자
```

```
    nicknames = soup.select('span.u_cbox_nick')
    list_nicknames = [nickname.text for nickname in nicknames]

    # 2)댓글 시간
    datetimes = soup.select('span.u_cbox_date')
    list_datetimes = [datetime.text for datetime in datetimes]

    # 3)댓글 내용
    contents = soup.select('span.u_cbox_contents')
    list_contents = [content.text for content in contents]



    # 4)작성자, 댓글 시간, 내용을 셋트로 취합
    list_sum = list(zip(list_nicknames,list_datetimes,list_contents))

    # 드라이버 종료
    driver.quit()

    # 함수를 종료하며 list_sum을 결과물로 제출
    return list_sum

# step3. 실제 함수 실행 및 엑셀로 저장
if __name__ == '__main__': # 설명하자면 매우 길어져서 그냥 이렇게 사용하는 것을 권장

    # 원하는 기사 url 입력
    url = '댓글 크롤링 원하는 기사의 url (주의! 댓글보기를 클릭한 상태의 url이어야 함'

    # 함수 실행
    comments = get_naver_news_comments(url)

    # 엑셀의 첫줄에 들어갈 컬럼명
    col = ['작성자','시간','내용']

    # pandas 데이터 프레임 형태로 가공
    df = pd.DataFrame(comments, columns=col)

    # 데이터 프레임을 엑셀로 저장 (파일명은 'news.xlsx', 시트명은 '뉴스 기사 제목')
    df.to_excel('news.xlsx', sheet_name='뉴스 기사 제목')
```

실시간으로 네이버 뉴스를 텔레그램으로 전송받기

1. 프로그램 컨셉



2. 사용 라이브러리 정보

- requests (특정 url의 html 문서 받기)
- BeautifulSoup (html 문서에서 원하는 요소 선택적 추출)
- python-telegram-bot (뉴스 링크를 텔레그램 봇으로 채팅방에 전송)

3. 전체 코드

이해하기 쉽도록 자세하게 주석을 달아두었으니 어렵지 않을거예요.

참고로 이 코드가 담긴 파이썬 파일(.py)을 pyinstaller를 이용해서 exe 파일로 만들어서 윈도우 시작 시, 기본 프로그램으로 설정해 놓으면 나만의 알리미 프로그램을 만드실 수 있어요!

```
#step1.라이브러리 불러오기
import  requests
from  bs4  import  BeautifulSoup  as  bs
import  telegram
import  schedule
import  time

#step2.새로운 네이버 뉴스 기사 링크를 받아오는 함수
def  get_new_links(old_links=[]):

    # (주의) 네이버에서 키워드 검색 - 뉴스 탭 클릭 - 최신순 클릭 상태의 url
    url  = f'https://search.naver.com/search.naver?where=news&query={query}&sm=tab_opt&sort=1&
photo=0&field=0&pd=0&ds=&de=&docid=&related=0&mynews=0&office_type=0&office_section_code=0&new
s_office_checked=&nso=so%3Add%2Cp%3Aall&is_sug_officeid=0'

    # html 문서 받아서 파싱(parsing)
    response  =  requests.get(url)
    soup  =  bs(response.text , 'html.parser')

    # 해당 페이지의 뉴스기사 링크가 포함된 html 요소 추출
    news_titles  =  soup.select('a.news_tit')

    # 요소에서 링크만 추출해서 리스트로 저장
    list_links  = [i.attrs['href'] for  i  in  news_titles]

    # 기존의 링크와 신규 링크를 비교해서 새로운 링크만 저장
    new_links  = [link  for  link  in  list_links  if  link  not  in  old_links]

    return  new_links


#step3.새로운 네이버 뉴스 기사가 있을 때 텔레그램으로 전송하는 함수
def  send_links():
    # 함수 내에서 처리된 리스트를 함수 외부에서 참조하기 위함
    global old_links

    # 위에서 정의했던 함수 실행
    new_links  =  get_new_links(old_links)

    # 새로운 메시지가 있으면 링크 전송
    if  new_links:
        for  link  in  new_links:
```

```
            bot.sendMessage(chat_id=chat_id, text=link)

    # 없으면 패스
    else:
        pass

    # 기존 링크를 계속 축적하기 위함

    old_links +=  new_links.copy()


# 실제 프로그램 구동
if  __name__  ==  '__main__':

    #토큰을 변수에 저장
    bot_token  ='자신이 발급받은 봇의 토큰'
    bot  =  telegram.Bot(token  =  bot_token)

    #가장 최근에 온 메세지의 정보 중, chat id만 가져옴 (이 chat id는 사용자(나)의 계정 id임)
    chat_id  =  bot.getUpdates()[-1].message.chat.id

    #step4.검색할 키워드 설정
    query  =  input('크롤링 할 뉴스기사 키워드를 입력하세요: ')

    #위에서 얻은 chat id로 bot이 메세지를 보냄.
    bot.sendMessage(chat_id  =  chat_id, text=f"{query}를 주제로 뉴스 기사 크롤링이 시작 되었습
니다")

    #step5.기존에 보냈던 링크를 담아둘 리스트 만들기
    old_links  = []

    # 주기적 실행과 관련된 코드 (hours는 시, minutes는 분, seconds는 초)
    job  =  schedule.every(10).seconds.do(send_links)

    while  True:
        schedule.run_pending()
        time.sleep(1)
```

4. 여러개의 검색어 크롤링하기

```
#step1.라이브러리 불러오기
import  requests
from  bs4  import  BeautifulSoup  as  bs
import  telegram
import  schedule
import  time

# step2.새로운 네이버 뉴스 기사 링크를 받아오는 함수

def get_new_links(query, old_links=[]):

    # (주의) 네이버에서 키워드 검색 - 뉴스 탭 클릭 - 최신순 클릭 상태의 url
    url = f'https://search.naver.com/search.naver?where=news&query={query}&sm=tab_opt&sort=1&p
hoto=0&field=0&pd=0&ds=&de=&docid=&related=0&mynews=0&office_type=0&office_section_code=0&news
_office_checked=&nso=so%3Add%2Cp%3Aall&is_sug_officeid=0'

    # html 문서 받아서 파싱(parsing)
    response = requests.get(url)
    soup = bs(response.text, 'html.parser')

    # 해당 페이지의 뉴스기사 링크가 포함된 html 요소 추출
    news_titles = soup.select('a.news_tit')

    # 요소에서 링크만 추출해서 리스트로 저장
    list_links = [i.attrs['href'] for i in news_titles]

    # 기존의 링크와 신규 링크를 비교해서 새로운 링크만 저장
    new_links = [link for link in list_links if link not in old_links]

    return new_links


# step3.새로운 네이버 뉴스 기사가 있을 때 텔레그램으로 전송하는 함수
def send_links(query):
    # 함수 내에서 처리된 리스트를 함수 외부에서 참조하기 위함
    global old_links

    # 위에서 정의했던 함수 실행
```

```
    new_links = get_new_links(query, old_links)

    # 새로운 메시지가 있으면 링크 전송
    if new_links:
        bot.sendMessage(chat_id=chat_id, text='방금 업데이트 된 ' + f"{query} 주제의 크롤링입니다.")
        for link in new_links:
            bot.sendMessage(chat_id=chat_id, text=link)

    # 없으면 패스
    else:
        pass

    # 기존 링크를 계속 축적하기 위함

    old_links += new_links.copy()


# 실제 프로그램 구동
if __name__ == '__main__':

    # 토큰을 변수에 저장
    bot_token = '자신이 발급받은 봇의 토큰'
    bot = telegram.Bot(token=bot_token)

    # 가장 최근에 온 메세지의 정보 중, chat id만 가져옴 (이 chat id는 사용자(나)의 계정 id임)
    chat_id = bot.getUpdates()[-1].message.chat.id


    # #step4.검색할 키워드 설정
    # query  =  input('크롤링 할 뉴스기사 키워드를 입력하세요: ')
    queries = ["부동산", "경제", "날씨"]

    for query in queries:

        # 위에서 얻은 chat id로 bot이 메세지를 보냄.
        bot.sendMessage(chat_id=chat_id,
                        text=f"{query}를 주제로 뉴스 기사 크롤링이 시작 되었습니다")

        # step5.기존에 보냈던 링크를 담아둘 리스트 만들기
        old_links = []

        # 주기적 실행과 관련된 코드 (hours는 시, minutes는 분, seconds는 초)
```

```
        job = schedule.every(10).seconds.do(send_links, query)

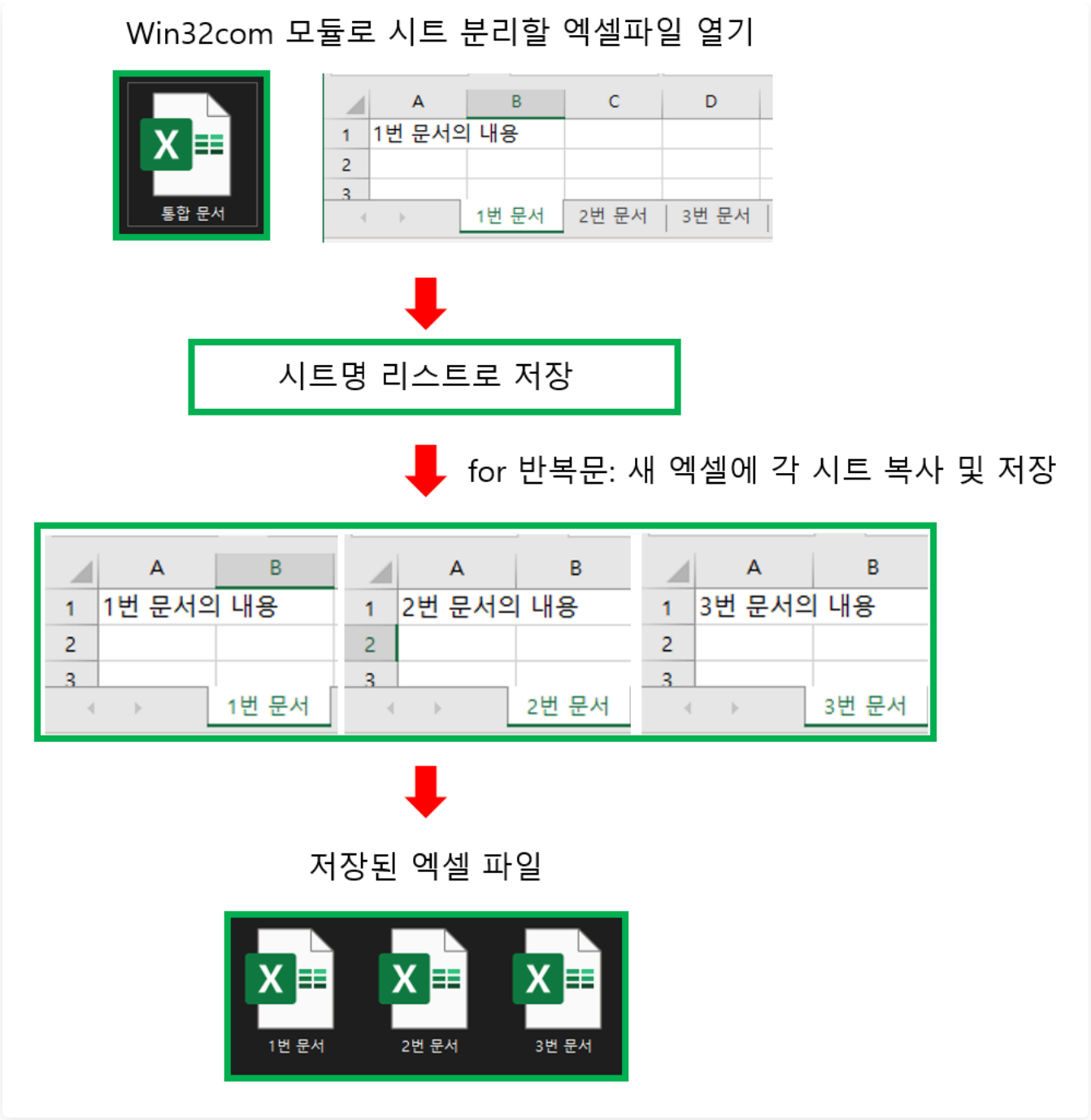
    while True:
        schedule.run_pending()
        time.sleep(1)
```

엑셀 시트를 개별 파일로 분리하기

1. 프로그램 컨셉

이번 시간에는 하나의 엑셀 파일에 있는 엑셀 시트를 각각 분리해서 엑셀 파일로 저장해보는 프로젝트를 해보겠습니다.

컨셉은 다음과 같습니다.



2. 사용 라이브러리 정보

- Pywin32 패키지의 win32com 모듈 (엑셀 어플리케이션을 다루기 위함)

3. 전체 코드

이해하기 쉽도록 자세하게 주석을 달아두었으니 어렵지 않을거예요. 만약 이해가지 않으시는 부분은 댓글로 질문해주세요!

```
# step1.관련 모듈 및 패키지 import
import win32com.client

# step2.win32com(pywin32)를 이용해서 엑셀 어플리케이션 열기
excel = win32com.client.Dispatch("Excel.Application")
excel.Visible = True #실제 작동하는 것을 보고 싶을 때 사용

# step3. 분리할 엑셀 파일이 있는 폴더 경로
path = r"C:\Users\SANGWOO\Desktop\VSCODE\엑셀 시트 분리"+"/"

# step4.분리하고자 하는 엑셀 통합문서 가져오기
wb = excel.Workbooks.Open(path + "통합 문서.xlsx")

# step5.통합문서의 엑셀 시트들의 이름을 리스트로 받기
Sheet_list = [sheet.Name for sheet in wb.Sheets]

# step6. 엑셀 시트 분리 및 저장 반복문
for sheet in Sheet_list:

    # 시트를 분리 저장할 새로운 엑셀 파일 open
    wb_new = excel.Workbooks.Add()

    # 통합문서의 시트 하나를 새로운 엑셀 문서에 copy
    wb.Worksheets(sheet).Copy(Before=wb_new.Worksheets("Sheet1"))

    # 처음 새로운 엑셀파일이 생성될 때 존재하는 "Sheet1" 시트 삭제
    wb_new.Worksheets("Sheet1").Delete()

    # 기존 경로에 시트명을 파일명으로 엑셀 파일 저장
    wb_new.SaveAs(path + sheet + ".xlsx")

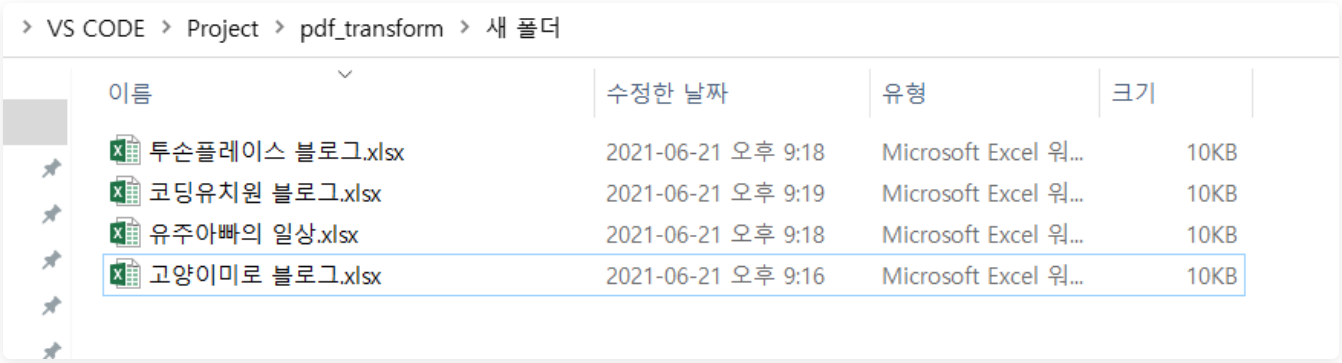
# step7. 켜져있는 엑셀 및 어플리케이션 모두 종료
excel.Quit()
```

엑셀 시트별 PDF 자동 저장하기

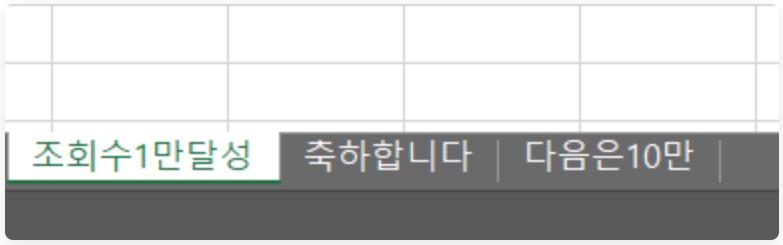
1. 프로그램 컨셉

이번 편에서는 엑셀 파일의 모든 시트를 개별적을 pdf 파일로 저장하는 기능을 가진 프로그램에 대해 코드를 작성해보겠습니다. 먼저 코드를 설명하기 전에 프로그램 컨셉에 대해 간단히 설명하겠습니다.

Concept 1. 한 폴더에 여러 엑셀파일이 존재합니다.



Concept 2. 1개 엑셀파일에는 다수의 시트가 존재합니다.

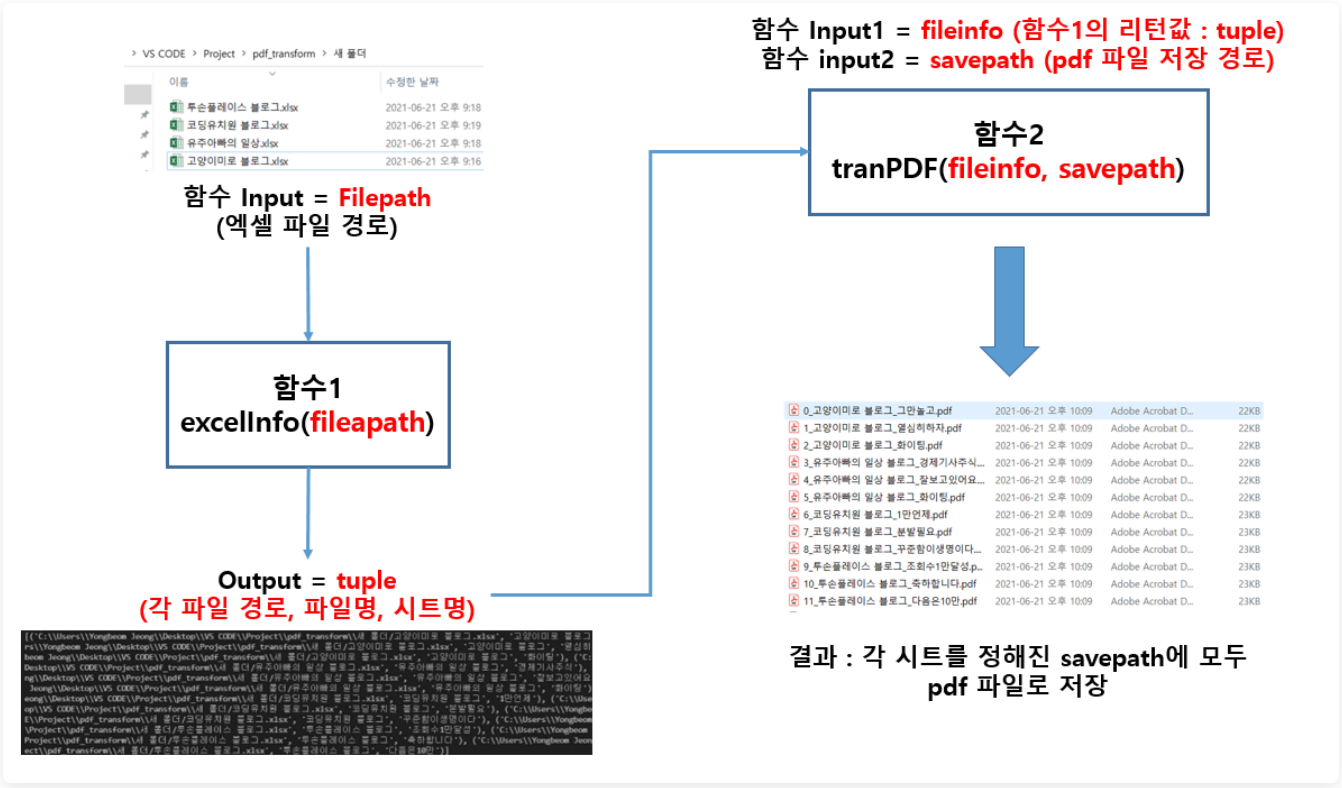


Concept 3. 프로그램 실행시 모든 엑셀파일의 시트를 개별적으로 pdf로 저장하게 됩니다.

0_고양이미로 블로그_그만놀고.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	22KB
1_고양이미로 블로그_열심히하자.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	22KB
2_고양이미로 블로그_화이팅.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	22KB
3_유주아빠의 일상 블로그_경제기사주식....	2021-06-21 오후 10:09	Adobe Acrobat D...	22KB
4_유주아빠의 일상 블로그_잘보고있어요....	2021-06-21 오후 10:09	Adobe Acrobat D...	22KB
5_유주아빠의 일상 블로그_화이팅.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	22KB
6_코딩유치원 블로그_1만언제.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	23KB
7_코딩유치원 블로그_분발필요.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	23KB
8_코딩유치원 블로그_꾸준함이생명이다....	2021-06-21 오후 10:09	Adobe Acrobat D...	23KB
9_투손플레이스 블로그_조회수1만달성.p...	2021-06-21 오후 10:09	Adobe Acrobat D...	23KB
10_투손플레이스 블로그_축하합니다.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	23KB
11_투손플레이스 블로그_다음은10만.pdf	2021-06-21 오후 10:09	Adobe Acrobat D...	23KB

위 컨셉을 종합하여 그려본 프로그램 개념도는 아래와 같습니다. 엑셀의 시트 정보를 읽어오는 excelInfo 함수와 excelInfo 함수 결과값을 활용해 pdf 파일로 변환하는 transPDF 함수를 통해 프로그램을 구현하게 됩니다.

※ 프로그램 개념도



2. 사용 라이브러리 정보

이번 챕터에서는 엑셀을 다루는 패키지인 OpenPyxl과 win32com 라이브러리를 사용합니다.

- OpenPyxl : 엑셀의 시트 정보를 읽어올 때 사용(Ch3-1 참고)
- win32com.client : 엑셀을 pdf로 저장할 때 사용 (Ch3-3 참고)
- os 모듈 : 폴더 안의 파일 정보를 읽어올 때 사용 (Ch3-4 참고)

3. 전체 코드

코드에 대한 설명은 주석으로 달아놓았으니 아래 내용을 참고해주시기 바랍니다.

```
import win32com.client
import openpyxl as op
import os


def excelInfo(filepath):
    excel_list = os.listdir(filepath) #폴더안에 잇는 엑셀파일 명을 리스트로 저장
    result = [] #빈 리스트 생성
    for file in excel_list: #엑셀파일명 리스트를 for문을 통해 반복
        wb = op.load_workbook(filepath+"/"+file) #openpyxl workbook 생성
        ws_list = wb.sheetnames #해당 workbook의 시트명을 리스트로 받음
        filename = file.replace(".xlsx","") #파일명을 저장하기 위해 문자열에서 확장자 제거
        for sht in ws_list: #시트명 리스트를 for문을 통해 반복
            temp_tuple = (filepath+"/"+file, filename, sht) #파일경로, 파일명, sht를 튜플에 저장
            result.append(temp_tuple) #위 튜플을 빈 리스트에 추가
    print(result)
    return result # 튜플로 이루어진 리스트 리턴


def transPDF(fileinfo, savepath):

    excel = win32com.client.Dispatch("Excel.Application")
    i=0 #파일명 중복을 방지하기 위한 인덱싱 번호

    #excelinfo를 입력받아 for문 실행
    for info in fileinfo:
        wb = excel.Workbooks.Open(info[0]) #info가 튜플이므로 인덱싱으로 접근(0번째는 파일경로)
        ws = wb.Worksheets(info[2]) #튜플의 2번째 요소는 시트명임.

        ws.Select() #위 설정한 시트 선택
        wb.ActiveSheet.ExportAsFixedFormat(0, savepath+"/"+str(i)+"_"+info[1]+"_"+info[2]+".pd
f") #파일명, 시트명으로 pdf 파일명 저장
        i=i+1
        wb.Close(False) #workbook 닫기, True일 경우 그 상태를 저장한다.
        excel.Quit()  # excel application 닫기


#실행부
if __name__ == "__main__"
    filepath = "엑셀파일 경로"
    pdfpath = "pdf파일을 저장할 경로"
```

```
    #excelinfo 실행(결과값은 튜플임)
    excelinfo = excelInfo(filepath)

    #transPDF 실행(실제 pdf파일로 변환하는 부분)
    transPDF(excelinfo, pdfpath)
```

엑셀의 교육자 명단을 참조하여 수료증(워드) 자동 작성하기

수료증 발급 자동화

외부 교육을 듣고 나면 제출용으로 수료증을 발급해주는 경우가 있습니다. 교육 참가자가 적다면 워드 양식을 켜서 한명씩 수정하면서 발급이 가능할테지만, 만약 교육 규모가 크고 자주 있다면 교육 담당자는 수료증을 발급하는 업무만 해도 많은 시간을 소모할 것입니다.

이번 연습하기 시간에는 이번 챕터에서 배운 지식과 지난 액셀 챕터에서 배운 내용을 토대로 '액셀 시트의 교육 참가자 명단을 참고해서 수료증을 자동으로 작성하고 저장'하는 것을 해보겠습니다.

우선 Word 양식을 불러와서 구조를 파악해봅시다. 보통은 각자의 회사에서 사용하는 양식이 이미 있는 경우가 대부분이기 때문에, 양식을 A부터 Z까지 코드로 구현하는 것은 생략하겠습니다.

저는 아래와 같은 양식을 만들어서 '교육 수료증.docx'라는 파일명으로 저장하였습니다.

수 료 증

성 명: 홍길동

생년월일: 1900-01-13

수료기간: 2022-03-15 ~ 2022-03-19

위 사람은 "사장님 몰래하는 파이썬 업무자동화"

과정을 성실하게 수료하였으므로 이 증서를 드립니다.

2022년 8월 15일

OO회사 대표 XXX

step1. 가장 처음으로 해줄 작업은 양식의 구조를 파악하는 일입니다. 구조 파악을 위해 아래의 코드를 실행해보세요.

> *코드*

```
from docx import Document

doc = Document('인덱싱 예제 문서.docx')

for i, paragraph in  enumerate(doc.paragraphs):

print(str(i)  +  ": "  + paragraph.text)
```

> *결과*

```
0: 수 료 증
1:
2:
3: 성 명: 홍길동
4: 생년월일: 1900-01-13
5: 수료기간: 2022-03-15 ~ 2022-03-19
6:
7:
8:
9: 위 사람은 “사장님 몰래하는 파이썬 업무자동화”
10: 과정을 성실하게 수료하였으므로 이 증서를 드립니다.
11:
12:
13:
14:
15:
16:
17:
18:
19:
20:
21: 2022년 8월 15일
22: OO회사 대표 XXX
23:
```

step2. 바꿔 줄 부분을 정하고, 다른 정보로 수정해보겠습니다. 저는 같은 차수의 교육을 들은 교육생들의 수료증을 발급해주는 상황을 가정하고 성명과 생년월일만 바꿔주어 보겠습니다.

> *코드*

```
from docx import Document

doc = Document('교육 수료증.docx')

#스타일 적용하기 (일반 텍스트, 글꼴은 '맑은고딕체')
from docx.oxml.ns import qn
style = doc.styles['Normal']
style.font.name =  '맑은 고딕'

style._element.rPr.rFonts.set(qn('w:eastAsia'), '맑은 고딕')

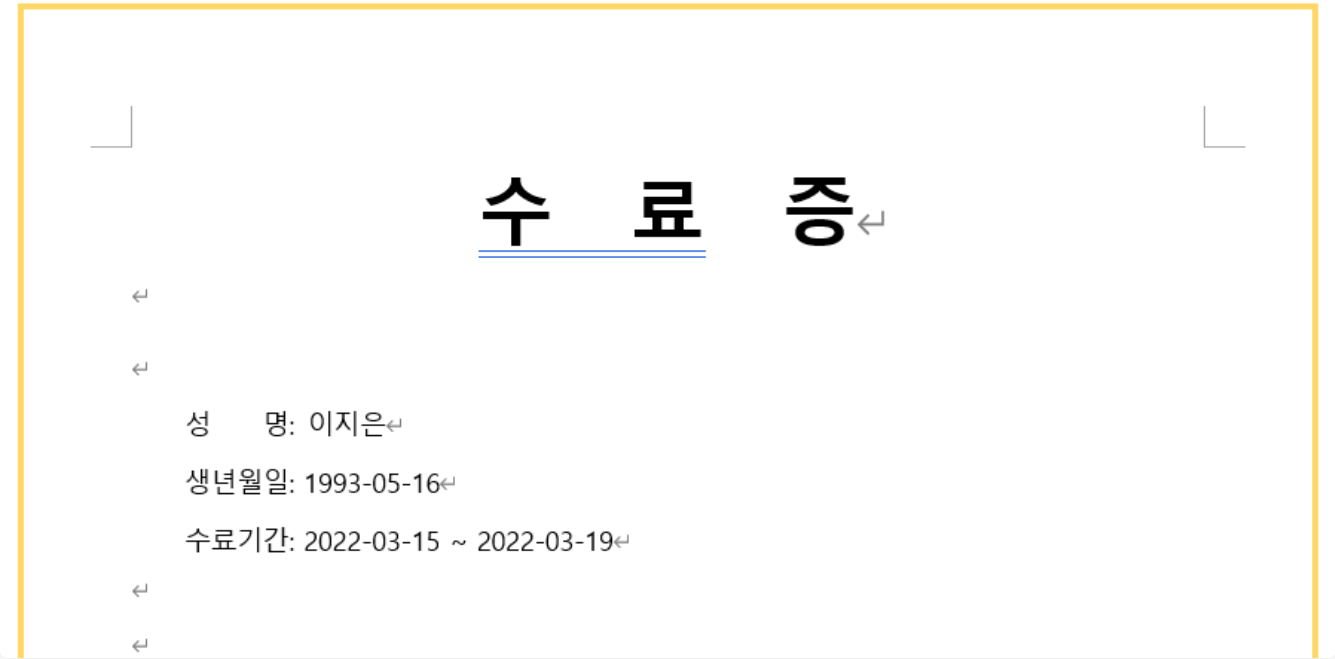
# 성명 수정
p = doc.paragraphs[3]
p.text = '성 명: 이지은'

# 생년월일 수정
p = doc.paragraphs[4]
p.text = '생년월일: 1993-05-16'

#저장
doc.save('교육 수료증_아이유.docx')
```

> *결과*

아래와 같이 홍길동에서 이지은으로 성명과 생년월일이 변경된 것을 확인하실 수 있습니다.



수 료 증

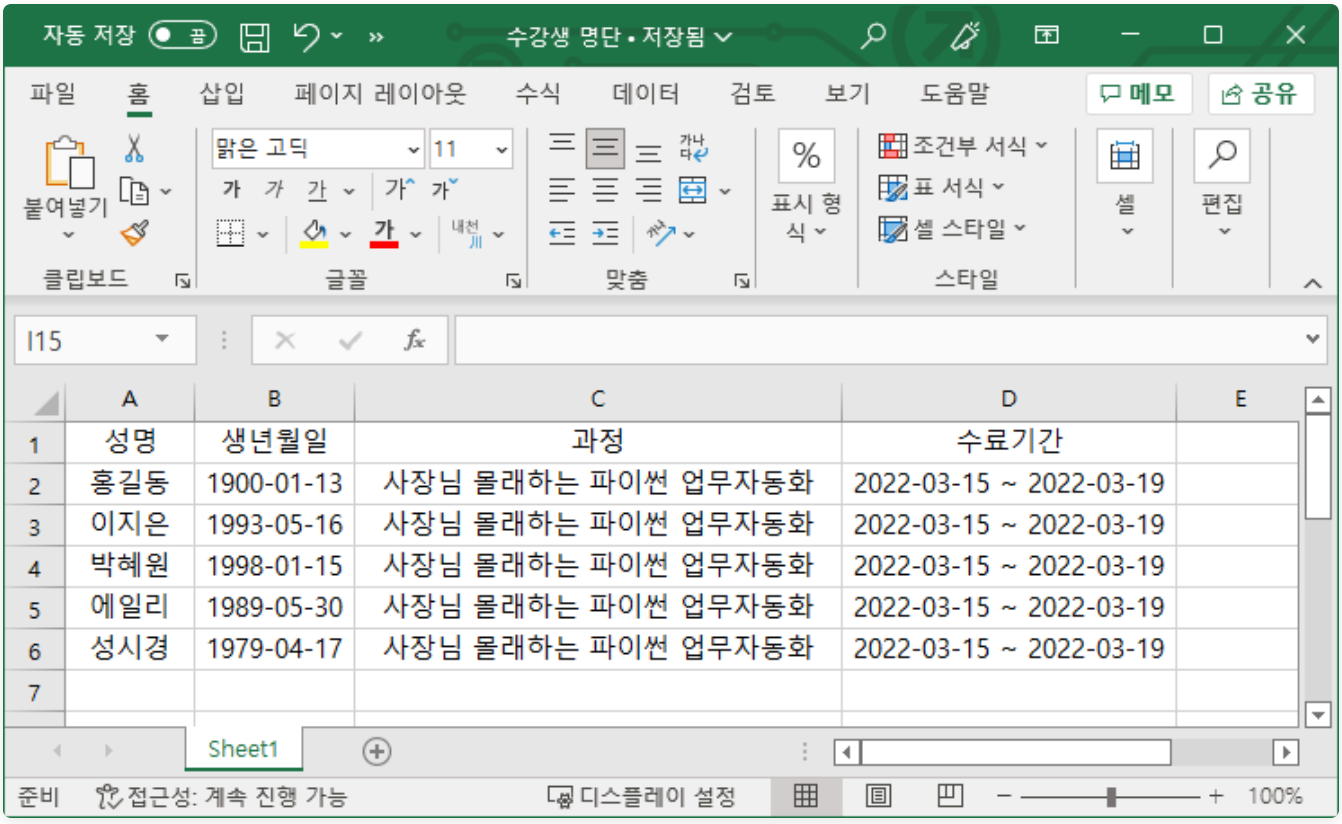
성 명: 이지은

생년월일: 1993-05-16

수료기간: 2022-03-15 ~ 2022-03-19

step3. Word 양식과 내용 변경 방법을 알아보았으니, 엑셀 리스트와 연동할 차례입니다. 수강생들의 리스트가 엑셀로 잘 정리되어 있다고 가정하고 진행해보겠습니다.

Word 파일에 바로 입력하기 전에 쉬운 이해를 돕기 위해서 명단이 포함된 엑셀 파일에서 성명과 생년월일을 순서대로 불러와서 print( )함수로 출력시켜 보겠습니다.



	A	B	C	D	E
1	성명	생년월일	과정	수료기간	
2	홍길동	1900-01-13	사장님 몰래하는 파이썬 업무자동화	2022-03-15 ~ 2022-03-19	
3	이지은	1993-05-16	사장님 몰래하는 파이썬 업무자동화	2022-03-15 ~ 2022-03-19	
4	박혜원	1998-01-15	사장님 몰래하는 파이썬 업무자동화	2022-03-15 ~ 2022-03-19	
5	에일리	1989-05-30	사장님 몰래하는 파이썬 업무자동화	2022-03-15 ~ 2022-03-19	
6	성시경	1979-04-17	사장님 몰래하는 파이썬 업무자동화	2022-03-15 ~ 2022-03-19	
7					

코드

```
from openpyxl import load_workbook

wb = load_workbook("수강생 명단.xlsx")
ws = wb.active

for i in  range(ws.max_row):
    name = ws.cell(row=i+1,  column=1).value
    birth = ws.cell(row=i+1,  column=2).value

    if i ≠  0: # 가장 첫줄(성명, 생년월일) 제거

        print(name, str(birth)[:10]) #datetime 형식의 생년월일 데이터를 원하는 형태로 변환
```

결과

```
홍길동 1900-01-13
이지은 1993-05-16
박혜원 1998-01-15
에일리 1989-05-30
성시경 1979-04-17
```

step4. 엑셀에서 원하는 데이터를 가져왔으니, 차례대로 수료증을 작성해봅시다.

코드

```
from docx import Document
from docx.oxml.ns import qn
from openpyxl import load_workbook

#스타일 적용하기 (일반 텍스트, 글꼴은 '맑은고딕체')
style = doc.styles['Normal']
style.font.name =  '맑은 고딕'
style._element.rPr.rFonts.set(qn('w:eastAsia'), '맑은 고딕')

doc = Document('교육 수료증.docx')
wb = load_workbook("수강생 명단.xlsx")

ws = wb.active

for i in  range(ws.max_row):
    name = ws.cell(row=i+1,  column=1).value
    birth = ws.cell(row=i+1,  column=2).value

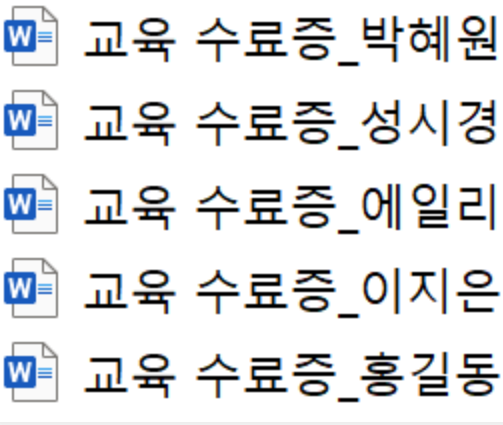
    if i ≠  0:

        # 성명 수정
        p = doc.paragraphs[3]
        p.text =  '성 명: '  + name

        # 생년월일 수정
        p = doc.paragraphs[4]
        p.text =  '생년월일: '  +  str(birth)[:10]

    #저장
    doc.save('교육 수료증_'  + name +  '.docx')
```

> *결과*


교육 수료증_박혜원
교육 수료증_성시경
교육 수료증_에일리
교육 수료증_이지은
교육 수료증_홍길동

잡플래닛 리뷰 정보 크롤링 & 엑셀 저장 (2024.08.18 수정)

1. 프로젝트 목적

이 프로젝트의 목적은 회사의 경영진들에게 직원들의 솔직한 의견들을 전달하기 위하여 수백건의 리뷰를 엑셀 파일로 정리하는 것입니다.

참고로 이 코드를 이용하려면 잡플래닛 계정이 있고, 회사 리뷰를 최소 1개 작성해야 합니다. 리뷰를 작성해야지, 모든 리뷰가 조회 가능하거든요.

<네이버랩스 잡플래닛 리뷰 크롤링 예시>

	A	B	C	D	E	F	G	H	I	J
1		날짜	직무	재직여부	별점	요약	장점	단점	진에게 바라는 점	
2	0	2021. 02	IT/인터넷	전직원	5점	"연구중심의 ㅎ	연구중심 회사	안정적이지 못	안정적인 수익을 확보하는게	
3	1	2020. 12	엔지니어링	현직원	4점	"다니기좋은호	네이버에서 주	여러일을 하다	지금 너무 좋습니다저는 이회	
4	2	2020. 04	연구개발	현직원	5점	"대학원과 비슷	각 분야별 전듬	근무 시간이 ㄱ	현재 회사에 매우 만족 중입니	
5	3	2020. 04	IT/인터넷	현직원	5점	"개발자로 근무	자율적으로 업	특별히 떠오르	보상에 있어서 좀 더 신경 썼	
6	4	2020. 02	디자인	전직원	5점	"전반적으로 C	자유로운 분위	아무래도 IT 쪽	디자인 팀에 대한 지원이 조금	
7	5	2020. 01	인사/총무	전직원	4점	"오리역 바로	네이버의 기업	주변 인프라가	연구실과의 커뮤니케이션이 (	
8	6	2019. 12	IT/인터넷	현직원	4점	"로봇과 기타등	몇 년 전에 비	로봇 퍼스트로	신사옥 프로젝 끝나면 먹고 살	
9	7	2019. 07	IT/인터넷	전직원	4점	"눈치보지 않고	연차를 자유롭	자주 프로젝트	현실화 가능한 프로젝트를 책	
10	8	2019. 05	IT/인터넷	전직원	4점	"복지 여긴이 -	사무실 분위기	하지만 네이버	직원들과 보다 더 적극적으로	
11	9	2019. 04	IT/인터넷	현직원	5점	"똑똑하고 열심	합리적인 의사	아직 프로젝트	물론 인력이 부족하지만 채용	
12	10	2019. 04	IT/인터넷	현직원	4점	"다양한 직군들	복지도 좋고 ㄱ	회사의 방향성	회사의 문화를 지금과 같이 유	
13	11	2019. 02	디자인	현직원	2점	"네이버의 별루	수평적이고 자	대외적으로 알	새로운 대표 체제에서도 부디	
14	12	2019. 02	IT/인터넷	전직원	5점	"국내에서 보기	사내에 식당이	자기주도적으.	단기적인 성과보다 좀 더 먼 (	
15	13	2019. 02	IT/인터넷	현직원	3점	"로보틱스, 자	수평적인 사무	매출을 내는 조	더이상 회사의 방향이 바뀌지	
16	14	2018. 09	IT/인터넷	현직원	4점	"뛰어난 동료들	네프님이 운영	회사의 전략이	전략이 바뀔때마다 커뮤니케(	
17	15	2018. 06	마케팅/시장조	전직원	5점	"보람있게 일할	프로젝트마다	아무래도 기술	근래에는 신입 채용도 해주셨	
18	16	2018. 04	IT/인터넷	현직원	4점	"새로운 기술이	연차뿐 아니라	최신 기술에 (	단계적으로 장기 계획을 수립	

2. 프로젝트에 사용된 라이브러리

Main

- selenium : 웹 조작, 웹 크롤링

Sub

- time : 타임 딜레이
- pandas : 리스트를 표 형태로 정리 및 엑셀로 저장

3. 코드

```
# step1.프로젝트에 필요한 패키지 불러오기
from  selenium  import  webdriver
from  selenium.webdriver.common.keys  import  Keys
from  selenium.webdriver.common.by  import  By
import  time
import  pandas  as  pd
import  math


# step2.로그인 정보 및 검색할 회사 미리 정의
USR  =  "잡플래닛 ID (e-mail address)"
PWD  =  "비밀번호"
QUERY  =  "네이버랩스"


# step3.크롬드라이버 실행 및 잡플래닛 로그인 함수
def  login(driver,  usr,  pwd):
    driver.get("https://www.jobplanet.co.kr/users/sign_in?_nav=gb")
    time.sleep(5)

    # 아이디 입력
    login_id  =  driver.find_element(By.ID,  "user_email")
    login_id.send_keys(usr)

    # 비밀번호 입력
    login_pwd  =  driver.find_element(By.ID,  "user_password")
    login_pwd.send_keys(pwd)

    # 로그인 버튼 클릭
    login_id.send_keys(Keys.RETURN)
    time.sleep(5)

# step4.원하는 회사의 리뷰 페이지까지 이동 함수

def  go_to_review_page(driver, query):

    # 검색창에 회사명 입력
    search_query  =  driver.find_element(By.ID, "search_bar_search_query")
```

```
    search_query.send_keys(query)
    search_query.send_keys(Keys.RETURN)
    time.sleep(3)

    # 회사명 클릭
    driver.find_element(
    By.CLASS_NAME, "line-clamp-2.text-h9.text-gray-800").click()
    time.sleep(2)

    # 새로운 창에 출력되므로 최신창으로 변환
    driver.switch_to.window(driver.window_handles[-1])
    time.sleep(5)

    # 리뷰 페이지 클릭
    try:
        driver.find_element(By.CLASS_NAME, "viewReviews").click()
    except:
        pass

    time.sleep(3)


# step5. 별점 변환 함수
def  parse_star_rating(style_attribute):
    if  len(style_attribute)  ==  11:
        rating_value  =  int(style_attribute[7:9])
        return  f"{rating_value  //  20}점"
    else:
        return  "5점"


# step6.데이터 크롤링 함수 (직무/근속여부/일시/요약/평점/장점/단점/경영진에게 바라는 점)
def  scrape_data(driver):

    list_div  =  []
    list_cur  =  []
    list_date  =  []
    list_stars  =  []
    list_summery  =  []
    list_merit  =  []
    list_disadvantages  =  []
    list_opinions  =  []
```

```
    # 크롤링 할 리뷰 갯수 파악
    review_count  =  driver.find_element(By.ID, "viewReviewsTitle")
    review_count  =  review_count.find_element(By.TAG_NAME, "span").text

    # 크롤링 할 페이지수 파악
    page  =  math.ceil(int(review_count) /  5)

    for  _  in  range(page):

        review_list  =  driver.find_element(By.ID, "viewReviewsList")
        review_box  =  review_list.find_elements(By.TAG_NAME, "section")

        # 페이지당 최대 5개의 리뷰 박스 존재
        for  i  in  review_box:

            user_info_area  =  i.find_elements(By.TAG_NAME, "div")[0]
            user_info  =  user_info_area.text.split("|")

            # 직무
            division  =  user_info[0]
            list_div.append(division)

            # 재직여부
            current  =  user_info[1]
            list_cur.append(current)

            # 날짜
            try:
                date  =  user_info[3]
                list_date.append(date)

            except: # 날짜 없는 경우 예외처리
                date  =  "날짜 없음"
                list_date.append(date)

            # 리뷰 요약
            try:
                summary  =  i.find_element(By.TAG_NAME, "h2")
                list_summery.append(summary.text)

            except: # 신고로 인해 리뷰 요약 없는 경우 예외처리
                # summery_ban = i.find_element(By.CLASS_NAME, "cont_discontinu.discontinu_cate
gory")
```

```
            summery_ban  =  "신고로 인해 리뷰 요약 없음"
            list_summery.append(summery_ban)
            list_merit.append(summery_ban)
            list_disadvantages.append(summery_ban)
            list_opinions.append(summery_ban)

        # 장점, 단점, 경영진에게 바라는 점
        try:
            review_info_area  =  i.find_elements(By.TAG_NAME, "dl")[-1]
            contents  =  review_info_area.find_elements(By.TAG_NAME, "dd")

            merit  =  contents[0].text
            list_merit.append(merit)

            disadvantage  =  contents[1].text
            list_disadvantages.append(disadvantage)

            opinion  =  contents[2].text
            list_opinions.append(opinion)

        except:
            pass

        try:
            star_info  =  i.find_elements(By.TAG_NAME, "span")

            has_star  =  False  # 별점이 있는지 확인하기 위한 플래그

            for  star  in  star_info:

                if  star.get_attribute("style") ≠  "": # 빈 값 제외

                    has_star  =  True

                    list_stars.append(
                    parse_star_rating(star.get_attribute("style"))
                    )

            if  not  has_star: # 별점이 없는 경우
                list_stars.append("별점 없음")

        except  Exception  as  e:
            print(f"별점 없음 - 예외 발생: {str(e)}")
```

```
                list_stars.append("별점 없음")

        try:
            driver.find_element(By.XPATH,'//*[@class="btn_pgnext inline-block h-[38px] w-[37p
x] bg-[url(https://common.jobplanet.co.kr/images/common/global_spt.png)] bg-[-96px_0] bg-no-re
peat hover:bg-[-96px_-48px]"]',).click()

            time.sleep(5)

        except:
            pass



    total_data  =  pd.DataFrame({

    '날짜':  list_date,
    '직무':  list_div,
    '고용 현황':  list_cur,
    '별점':  list_stars,
    '요약':  list_summery,
    '장점':  list_merit,
    '단점':  list_disadvantages,
    '경영진에게 바라는 점':  list_opinions

    })

    return  total_data



def  main():

    # 크롬 드라이버 실행
    driver  =  webdriver.Chrome()

    # 로그인
    login(driver,  USR,  PWD)

    # 리뷰 페이지로 이동
    go_to_review_page(driver,  QUERY)

    # 리뷰 클롤링
```

```
    total_data  =  scrape_data(driver)

    # 엑셀 파일로 저장
    total_data.to_excel(f"잡플래닛 리뷰 총정리_{QUERY}.xlsx",  index=True)

    # 크롬 드라이버 종료
    driver.close()



if  __name__  ==  "__main__":

    main()
```